ChatGPT

에듀도큐 매니저 (EduDocu Manager) 개발계획서

작성일: 2026년 2월 27일 **버전**: v1.0.1 (Google Sheets/AppSheet 개정판) **작성자**: 개발팀

목차

1. 프로젝트 개요
2. 기술 스택
3. 시스템 아키텍처
4. 주요 기능 명세
5. 데이터 모델 설계
6. 화면 구성 (UI/UX)
7. 개발 단계별 로드맵
8. API 연동 계획
9. 엑셀 출력 설계
10. 보안 및 데이터 관리
11. 확장 계획 (v2.0)
12. 위험 요소 및 대응 방안

---

1. 프로젝트 개요

1.1 앱 소개

에듀도큐 매니저는 방과후학교 강사가 학교별 학생·교구·수납 정보를 관리하고 필수 행정 서류(계획서, 출석부, 청구서 등)를 자동 생성하여 **업무 효율을 극대화**하는 **클라우드 기반 앱**입니다. 이번 개정판에서는 로컬 애플리케이션 대신 **Google Sheets**를 데이터베이스로 사용하고, **AppSheet**를 통해 웹·모바일 인터페이스를 제공하며, 복잡한 로직과 문서 생성은 **Google Apps Script**에서 수행합니다.

1.2 핵심 가치

항목	내용
☒목표	반복적인 행정 업무 자동화를 통한 강사 업무 부담 최소화
☒대상 사용자	방과후학교 담당 강사 (1인 또는 소규모 팀)
☒언어	한국어 인터페이스 및 한국어 엑셀 출력
☒데이터 저장	Google Sheets (클라우드 스프레드시트)

1.3 해결하는 문제

- 매월 반복되는 수업 일자 수동 계산 (공휴일 제외)
- 학교별 양식이 다른 서류의 수동 작성
- 수강 인원 변동 시 청구서 재계산 번거로움

- 연간 교육 계획의 월별 수동 분배

---

2. 기술 스택

2.1 플랫폼 & UI

분류	기술	선택 이유
앱 빌더	**AppSheet**	Google Sheets와 연동되는 노코드 플랫폼으로, 모바일·웹 UI를 자동 생성하고 복잡한 앱을 빠르게 구축 가능 [1].
설계 도구	**Google AI Studio Build**	자연어 설명으로 데이터 구조와 AppSheet 앱을 생성할 수 있어 초기 설계 시간을 단축한다.

2.2 데이터 & 스토리지

분류	기술	선택 이유
데이터베이스	**Google Sheets**	클라우드 기반 스프레드시트로, 실시간 협업과 권한 관리가 용이하며 적은 규모의 데이터를 효과적으로 저장할 수 있다 [2].
파일 저장	**Google Drive**	서류 템플릿, 계획서, 지도안 등 파일을 중앙 저장소에 보관하고 URL을 Google Sheets에 기록한다.

2.3 자동화 & 백엔드

분류	기술	선택 이유
스크립트 언어	**Google Apps Script**	JavaScript 기반으로 Google Sheets, Drive, 외부 API를 제어할 수 있으며, 공휴일 API 호출, 데이터 계산, 서류 생성·엑셀 내보내기 등을 자동화한다 [1].
AI 모델	**Gemini API (Google AI Studio)**	계획서 분석, 자연어 입력 변환 등 AI 기능을 구현한다.

2.4 외부 API

분류	기술	비고
공휴일 API	**한국천문연구원 특일 정보 API (data.go.kr)**	법정·임시 공휴일 자동 수집; Apps Script에서 호출한다.

---

3. 시스템 아키텍처

```
┌──────────────
```

```
│   │  사용자 인터페이스      │   │  액션 & 자동화 (Bots)      │    │
│   │  - 폼, 테이블, 대시보드 │   │  - 문서 생성 트리거        │    │
│   │  - 데이터 입력/수정     │   │  - 알림 전송               │    │
│   └──────────────────────────────────────┘
└──────────────────────────────────────────────┘    │
│              │                          │          │
│              ▼                          ▼          │
│           Google Sheets ←──────────── Apps Script ──────┐
│          (데이터 스토리지)     (비즈니스 로직/문서 생성) │ │
│                                         │            │ │
│                              ┌──────────────▼──────────────┐    │ │
│                              │ 외부 API 호출      │    │ │
│                              │ - 공휴일 API       │    │ │
│                                  │ - Gemini API       │    │ │
│                              └──────────────────────────────┘    │ │
└──────────────────────────────────────────────────────────────────────────────
```

3.1 데이터 흐름

```
[사용자(AppSheet)] → [AppSheet 테이블] → [Google Sheets]
                                ↓
                        [Apps Script 트리거 실행]
                                ↓
       [공휴일 API] → [Apps Script 날짜 엔진] → [시트 업데이트]
                                ↓
                [문서 생성 & Google Drive 저장]
                                ↓
                      [엑셀 다운로드 링크 제공]
```

---

4. 주요 기능 명세

4.1 마스터 데이터 관리

AppSheet의 폼과 테이블 뷰를 활용하여 모든 마스터 데이터를 Google Sheets에 저장한다. 각 시트는 AppSheet에서 `Ref` 타입으로 관계가 설정된다.

- **학교 관리**: 학교명, 수업 요일(복수 선택), 수업 시작 시간, 1인당 수강료, 담당 교사 이름/연락처, 서류 템플릿 링크를 입력할 수 있다. 학교별로 여러 반을 구성할 수 있다.
- **반(클래스) 관리**: 반 이름, 소속 학교(Ref), 담당 교사, 반 설명 등을 기록한다.
- **학생 관리**: 학생 이름, 학년, 소속 반(Ref), 등록일, 수강 시작월, 수강 상태(활성/비활성), 교구비 포함 여부, 교구 상태(새·기존·미사용), 보호자 연락처를 관리한다.
- **커리큘럼(교구) 관리**: 커리큘럼 조합명과 교구 유형(항공·안전망·코딩 등)을 정의하고, 차시별 제목·내용·재료·지도안/계획서 파일 링크를 추가한다. 차시 순서를 번호(`session_no`) 컬럼으로 관리하여 AppSheet에서 쉽게 정렬할 수 있다.
- **출석 및 수납 관리**: 수업 날짜별 학생 출석 상태(출석, 결석, 보강)와 메모를 기록하고, 각 학생의 연·월별 수강료·교구비 납부 여부와 납부일을 기록한다.

4.2 스마트 날짜 계산

수업 일정은 Apps Script가 한국천문연구원 공휴일 API를 호출하여 월별 공휴일 목록을 구한 뒤, 학교의 요일 설정과 비교하여 자동으로 계산한다. 기본 알고리즘은 다음과 같다:

```
입력: 학교 수업 요일(예: 화·목), 대상 연월, 공공데이터 공휴일 목록
처리:
 1. 해당 월의 날짜를 나열하고 수업 요일에 해당하는 날짜만 필터링
 2. 공휴일 날짜와 겹치는 날은 제외
 3. 사용자가 추가·제외할 수 있도록 AppSheet 편집 기능 제공
출력: 실제 수업 일자 리스트 (예: 3/3, 3/5, 3/10 …)
```

계산 결과는 Google Sheets의 특정 시트에 저장되며, 이후 문서 생성에 사용된다.

4.3 연간 차시 시퀀싱

각 학교·반별로 누적 수업 회차를 관리하여 해당 월의 수업 횟수만큼 다음 차시를 자동 배정한다. Apps Script는 이전 월의 마지막 차시를 확인하고 새로운 차시 번호를 산출하여 커리큘럼 시트를 순차적으로 업데이트한다.

4.4 서류 자동 생성

Apps Script는 Google Drive에 저장된 템플릿을 불러와 수업 일정, 학생 명단, 출석 정보, 커리큘럼 내용 등을 삽입한 뒤 새 시트를 생성한다. 생성되는 서류 종류는 다음과 같으며, 시트 이름은 월과 반 이름을 조합하여 구성한다.

서류명	내용	시트 이름
월간계획서	수업 일자와 차시별 교육 주제·내용	`MM월-반명-계획서`
수강생 명단	학생 목록(가나다순)과 교구비 여부	`MM월-반명-명단`
출석부	학생 행 × 수업 날짜 열의 출석 상태	`MM월-반명-출석`
활동일지	월간계획서 내용을 세부 기술한 기록	`MM월-반명-활동`
강사료 청구서	수강 인원 × 단가 × 수업 횟수 계산	`MM월-반명-청구`

4.5 엑셀 내보내기

완성된 서류는 Google Sheets 안에서 각 시트로 생성된 뒤 Apps Script가 `export?format=xlsx` 엔드포인트를 호출하여 하나의 `.xlsx` 파일로 다운로드 링크를 제공한다 [3]. 서식(셀 병합·테두리·폰트 등)은 템플릿을 복사하면서 유지되고, 사용자는 AppSheet의 액션 버튼을 통해 언제든지 엑셀 파일을 받을 수 있다.

---

5. 데이터 모델 설계

5.1 ERD (Entity Relationship Diagram)

Google Sheets를 데이터베이스로 사용하지만, 각 시트는 명확한 관계를 갖는다. 주요 관계는 다음과 같다:

```
[Schools] 1 ─── N [Classes]
   │                  │
   │                 N [Students]
   │
   1 ─── N [CurriculumSets]
               │
                 1 ─── N [CurriculumItems]

[Students] 1 ─── N [Attendance]
[Students] 1 ─── N [Payments]
[Schools] + [Month] → [Documents]
```

각 시트의 행은 고유 ID를 갖고, 다른 시트에서 `Ref` 타입으로 참조된다.

5.2 시트별 컬럼 정의

Google Sheets의 각 시트는 다음과 같은 컬럼을 가진다. 설명은 간단한 키워드로 표기하며, AppSheet에서 필드 유형과 유효성 검사를 지정한다.

Schools

컬럼명	유형	설명
`id`	Text (Key)	고유 식별자 (AppSheet가 자동 생성)
`name`	Text	학교 이름
`days`	EnumList	수업 요일 (예: 화, 목)
`start_time`	Time	수업 시작 시간
`fee_per_student`	Number	1인당 수강료
`teacher_name`	Text	담당 교사 이름
`teacher_contact`	Text	담당 교사 연락처
`template_plan`	URL	월간계획서 템플릿 파일 링크
`template_roster`	URL	명단 템플릿 링크
`template_attendance`	URL	출석부 템플릿 링크
`template_activity`	URL	활동일지 템플릿 링크
`template_invoice`	URL	청구서 템플릿 링크

Classes

컬럼명	유형	설명
`id`	Text (Key)	반 ID

컬럼명	유형	설명
`school_id`	Ref → Schools	소속 학교
`class_name`	Text	반 이름
`teacher_name`	Text	반 담당 교사
`teacher_contact`	Text	반 담당 교사 연락처

Students

컬럼명	유형	설명
`id`	Text (Key)	학생 ID
`class_id`	Ref → Classes	소속 반
`name`	Text	학생 이름
`grade`	Text	학년
`is_active`	Yes/No	수강 상태
`include_kit`	Yes/No	교구비 포함 여부
`kit_status`	Enum	새교구/기존교구/미사용
`enrolled_at`	Date	수강 시작일
`parent_contact`	Text	보호자 연락처

CurriculumSets

컬럼명	유형	설명
`id`	Text (Key)	커리큘럼 세트 ID
`name`	Text	조합명 (예: 드론반 A)
`kit_type`	Enum	항공·안전망·코딩 등
`school_id`	Ref → Schools (선택)	특정 학교 전용일 경우

CurriculumItems

컬럼명	유형	설명
`id`	Text (Key)	차시 ID
`set_id`	Ref → CurriculumSets	소속 커리큘럼
`session_no`	Number	차시 번호
`title`	Text	수업 주제

컬럼명	유형	설명
`content`	Text	수업 내용
`materials`	Text	준비물
`plan_file_url`	URL	계획서 파일 링크
`guide_file_url`	URL	지도안 파일 링크

Attendance

컬럼명	유형	설명
`id`	Text (Key)	출석 기록 ID
`student_id`	Ref → Students	학생
`date`	Date	수업 날짜
`status`	Enum	출석/결석/보강
`memo`	Text	비고

Payments

컬럼명	유형	설명
`id`	Text (Key)	수납 ID
`student_id`	Ref → Students	학생
`year_month`	Text	연월 (예: 2026-03)
`tuition_paid`	Yes/No	수강료 납부 여부
`kit_paid`	Yes/No	교구비 납부 여부
`payment_date`	Date	납부일

Documents

컬럼명	유형	설명
`id`	Text (Key)	문서 ID
`school_id`	Ref → Schools	학교
`class_id`	Ref → Classes	반
`year_month`	Text	문서 생성 연월
`file_url`	URL	생성된 엑셀 파일의 Google Drive 링크
`created_at`	DateTime	생성 시각

---

6. 화면 구성 (UI/UX)

6.1 AppSheet 뷰 구성

AppSheet는 기본적으로 반응형 UI를 제공하므로 별도의 레이아웃 코딩 없이 뷰를 구성할 수 있다. 주요 뷰와 역할은 다음과 같다:

- **대시보드**: 요약 카드 형태로 학교 수, 반 수, 학생 수, 이번 달 서류 생성 여부, 다음 수업 일정 등을 표시한다. AppSheet의 Dashboard 뷰를 사용하여 여러 테이블(Attendance, Documents)을 한 화면에 배치할 수 있다.
- **Schools 뷰**: Deck 또는 Table 뷰로 학교 목록을 보여주고, 각 학교를 클릭하면 상세 정보와 하위 반 목록을 볼 수 있다. 폼을 통해 학교를 추가·수정·삭제할 수 있다.
- **Classes 뷰**: 학교를 선택하면 연관된 반 목록이 표시된다. 반 정보를 편집하고 담당 교사 정보를 입력할 수 있다.
- **Students 뷰**: 반을 선택하면 해당 반의 학생 목록이 나타난다. 학생 추가/수정/비활성화를 지원하며, Enum 및 Date 타입을 활용해 유효성을 검증한다.
- **CurriculumSets / Items 뷰**: 커리큘럼 세트와 차시 정보를 스프레드시트 형태로 편집할 수 있다. AppSheet의 Sheet view를 사용하면 차시 번호 정렬과 필터링이 가능하다. Gemini API 분석 결과를 반영하기 위해 Apps Script에서 데이터를 쓰고, AppSheet는 이를 자동으로 동기화한다.
- **Attendance 뷰**: 날짜별 출석 상태를 빠르게 입력할 수 있도록 Calendar 또는 Table 뷰를 사용한다. 기본 값은 '출석'으로 설정하고, 드롭다운으로 변경한다.
- **Payments 뷰**: 학생별 납부 현황을 월 단위로 표시하며, Yes/No 필드를 사용해 납부 여부를 체크한다.
- **Document Generator 액션**: 대시보드 또는 각 반의 상세 화면에서 "서류 생성" 버튼을 눌러 Apps Script 함수를 호출한다. 스텝별 마법사는 AppSheet의 Form 뷰로 구현할 수 있으며, 날짜 계산 결과를 미리보기로 보여준다.

6.2 미리보기 및 문서 확인

서류 생성 후 AppSheet는 `Documents` 시트에 파일 링크를 저장한다. 사용자는 목록 뷰에서 문서 이름을 클릭하여 Google Drive에 저장된 엑셀 파일을 바로 열 수 있으며, 문서마다 생성 일시와 상태를 확인할 수 있다. 문서 내용을 수정해야 할 경우, 원본 데이터(Students, CurriculumItems 등)를 변경한 뒤 다시 생성한다.

---

7. 개발 단계별 로드맵

AppSheet와 Google Sheets를 이용하면 전통적인 코딩보다 짧은 기간에 MVP를 구축할 수 있다. 예상 일정은 다음과 같다.

단계 1 — 데이터 모델링 및 스프레드시트 준비 (1주)

기간	작업 내용
1주차	Google Sheets 파일 생성, Schools/Classes/Students/Attendance/Payments/Curriculum/Document 시트 구조 설계 및 예시 데이터 입력

단계 2 — AppSheet 앱 구축 (2주)

기간	작업 내용
2주차	AppSheet 프로젝트 생성, 각 시트와 연동한 테이블/폼/대시보드 뷰 설정, 기본 액션 구현

기간	작업 내용
3주차	보안 필터 및 사용자 권한 설정, Dashboard 뷰와 Document Generator 액션 개발, 초기 사용자 테스트

단계 3 — Apps Script 자동화 개발 (2주)

기간	작업 내용
4주차	공휴일 API 연동 및 날짜 계산 함수 작성, 커리큘럼 시퀀싱 로직 및 문서 생성 스크립트 구현
5주차	Excel 다운로드 기능 구현 (`export?format=xlsx`), 템플릿 적용 로직 완성, API 키 저장/관리 로직 추가

단계 4 — AI 통합 및 고도화 (1주)

기간	작업 내용
6주차	Gemini API를 활용한 계획서 분석 및 커리큘럼 자동 입력 기능 개발, 오류 처리 및 UX 개선

단계 5 — 테스트 및 배포 (1주)

기간	작업 내용
7주차	실제 학교 데이터를 사용한 테스트, 사용자 피드백 반영, 사용 가이드 문서 작성 후 프로덕션 배포

총 예상 개발 기간: 약 7주 (1.5개월) — AppSheet를 사용하면 프론트엔드/백엔드 통합 개발 시간이 크게 단축된다.

---

8. API 연동 계획

8.1 공휴일 API 연동

- **제공처**: 행정안전부 / data.go.kr
- **API명**: 한국천문연구원 특일 정보
- **엔드포인트**: `https://apis.data.go.kr/B090041/openapi/service/SpcdeInfoService/getRestDeInfo`
- **요청 파라미터**: `solYear`, `solMonth`, `ServiceKey`
- **응답 형식**: XML (Apps Script에서 JSON 변환)
- **캐싱 전략**: Apps Script의 PropertiesService에 월별 응답을 저장하여 같은 월 요청 시 캐시를 사용하고, 24시간 후 갱신한다.
- **오프라인 대응**: API 호출이 실패하면 Google Sheets의 `Holidays` 시트에 수동으로 입력한 공휴일 목록을 사용한다.

8.2 Gemini API 사용

교육 계획서나 커리큘럼의 자연어 분석에는 Google AI Studio의 Gemini API를 사용한다. Apps Script가 HTTP 요청을 보내 분석 결과를 받아 커리큘럼 시트에 저장하며, AppSheet는 즉시 데이터를 반영한다.

8.3 API 키 관리

- **저장 위치**: Apps Script의 `PropertiesService.getScriptProperties()`에 저장하여 코드 외부에 숨긴다.
- **입력 방식**: 최초 실행 시 관리자가 AppSheet 설정 화면에서 키를 입력하면 Apps Script에서 받아 저장한다.
- **보안**: 키는 스크립트에서 직접 하드코딩하지 않고, Google Cloud Secret Manager 사용을 검토한다.

8.4 호출 예시 (Apps Script)

```
function getHolidays(year, month) {
  var key = PropertiesService.getScriptProperties().getProperty('HOLIDAY_API_KEY');
  var url = 'https://apis.data.go.kr/B090041/openapi/service/SpcdeInfoService/getRestDeInfo'
    + '?ServiceKey=' + key
    + '&solYear=' + year
    + '&solMonth=' + ('0' + month).slice(-2)
    + '&_type=json&numOfRows=30';
  var response = UrlFetchApp.fetch(url);
  var data = JSON.parse(response.getContentText()).response.body.items.item;
  return data;
}
```

---

9. 엑셀 출력 설계

9.1 시트 명명 규칙

Apps Script가 생성한 문서는 Google Sheets에서 각각의 시트로 저장된다. 시트 명명 규칙은 다음과 같다.

```
형식: MM월-반명[-서류종류]
예시:
 3월-A          (기본)
 3월-A-출석    (서류 구분 필요시)
 3월-드론반    (반 이름이 한글인 경우)
```

9.2 서류 레이아웃 가이드

서류 템플릿은 Google Drive에 MS Excel 형식으로 저장한 후 Apps Script가 복사하여 데이터를 삽입한다. 템플릿은 맑은 고딕 폰트, 얇은 테두리, 날짜·금액 형식 설정 등이 미리 지정되어 있어야 한다. 대표적인 레이아웃 예시는 다음과 같다.

- **월간계획서**: 상단에 학교명·반명·기간을 표시하고, 차시·날짜·주제·내용 열을 갖는다.
- **출석부**: 첫 열에 학생 이름을 세로로 나열하고, 이후 열에 수업 날짜를 헤더로 배치하여 출석 상태를 기록한다.
- **강사료 청구서**: 학교·반·연월 정보를 표시하고, 수강 인원, 1인당 수강료, 수업 횟수, 합계 금액을 계산한다.

9.3 공통 서식 설정

항목	설정값
기본 폰트	맑은 고딕, 10pt
제목 폰트	맑은 고딕 Bold, 12pt
셀 테두리	thin border (전체)
셀 정렬	헤더: 가운데, 내용: 왼쪽
날짜 형식	M/D (예: 3/5)
금액 형식	#,##0 원
페이지 설정	A4 세로, 여백 상하 10 mm / 좌우 10 mm

이 서식은 템플릿에 적용되어야 하며, Apps Script는 해당 서식을 유지한 채로 데이터를 삽입한다. 이후 사용자는 AppSheet 액션을 통해 엑셀 파일로 내려받을 수 있다.

---

10. 보안 및 데이터 관리

10.1 개인정보 보호 원칙

- **Google 계정 기반 인증**: AppSheet는 사용자 인증을 위해 Google 로그인을 사용하며, 데이터 접근은 공유된 Google Sheets 및 Drive 권한으로 제한된다.
- **데이터 최소화**: 학생 이름, 학년 등 필수 정보만 저장하고 주민등록번호 등 민감 정보는 수집하지 않는다.
- **행 수준 보안**: AppSheet의 보안 필터(Security Filter)를 활용하여 사용자별로 볼 수 있는 학교·반 데이터를 제한한다.

10.2 데이터 백업 및 복구

- **자동 백업**: Google Sheets는 변경 이력을 자동으로 저장하고, Google Drive의 버전 관리 기능을 통해 이전 버전으로 복구할 수 있다.
- **내보내기/가져오기**: 필요 시 전체 시트를 `.xlsx` 또는 `.csv`로 내보내어 별도 보관할 수 있으며, Google Sheets `import` 기능으로 복원할 수 있다.

10.3 API 키 및 시크릿 관리

- **스크립트 속성**: 외부 API 키는 Apps Script의 Script Properties 또는 Google Cloud Secret Manager에 저장한다.
- **입력 권한 제한**: 키를 입력하거나 수정하는 기능은 관리자 역할 사용자만 접근할 수 있다.
- **로그 노출 차단**: 키를 포함한 요청/응답은 로그에 기록하지 않으며, 예외 처리 시에도 키를 출력하지 않는다.

---

11. 확장 계획 (v2.0)

11.1 알림 및 자동화

- **서류 제출 알림**: AppSheet의 Bot을 이용하여 서류 제출 마감일 D-3, D-1에 이메일 또는 푸시 알림을 전송한다.

- **수업 일정 알림**: 일정 수 시간 전에 학생·강사에게 수업 시작 시간을 알리는 알림을 전송할 수 있다.
- **정기 보고서**: 매월 말 자동으로 강사료 청구서를 생성하고 발송하는 자동화 규칙을 설정한다.

11.2 사진 및 파일 첨부

- **활동 사진 첨부**: AppSheet의 파일 업로드 기능을 사용하여 활동일지에 사진을 첨부하고, Google Drive에 저장한다.
- **이미지 삽입**: Apps Script에서 Google Sheets API의 이미지 삽입 기능을 활용하여 활동일지 시트에 사진을 포함할 수 있다.

11.3 분석과 시각화

- **Looker Studio 연동**: Google Sheets의 데이터를 Looker Studio(구 Data Studio)와 연결하여 월별 강사료 수익, 학생 수 변동, 차시 진행 현황 등을 시각화하는 대시보드를 구축한다.
- **AppSheet 차트**: 간단한 추세 그래프는 AppSheet의 Chart 뷰로 제공하여 앱 내에서도 확인 가능하다.

11.4 문서 패키징 및 공유

- **ZIP 다운로드**: Apps Script로 여러 엑셀 파일을 하나의 ZIP 파일로 압축하여 한 번에 다운로드할 수 있게 한다.
- **공유 링크 관리**: 문서 생성 후 Google Drive의 공유 링크를 제한 공개로 설정하고, 필요한 사람에게만 링크를 전송한다.

11.5 오프라인 및 다중 기기 지원

- **오프라인 캐시**: AppSheet는 기본적으로 오프라인 캐싱을 제공하며, 연결이 복구되면 동기화가 이루어진다.
- **다중 기기**: Google 계정으로 로그인하면 모바일과 데스크톱에서 같은 데이터를 사용할 수 있다.

---

12. 위험 요소 및 대응 방안

위험 요소	영향도	대응 방안
공휴일 API 중단/변경	중	Apps Script에 기본 공휴일 목록을 내장하고, 수동 입력 기능을 제공하여 API 호출 실패 시에도 날짜 계산이 가능하도록 한다.
Google Sheets 셀 제한	중	데이터가 증가하면 하나의 스프레드시트가 1,000만 셀 제한에 도달할 수 있으므로, 학교별로 스프레드시트를 분리하거나 데이터를 아카이브하여 용량을 관리한다.
AppSheet 사용량/쿼터 제한	중	AppSheet의 요금제와 사용량 제한을 검토하고, 사용자 수 증가 시 적절한 플랜으로 업그레이드한다.
API 키 유출	높	Script Properties/Secret Manager를 사용하여 키를 안전하게 저장하고, 키 접근 권한을 관리자로 제한한다.
템플릿 다양성	중	학교별 템플릿을 Google Drive에 저장하고, Apps Script가 동적으로 선택하여 사용할 수 있도록 설계하여 다양한 양식을 대응한다.
네트워크 연결 문제	낮	AppSheet의 오프라인 기능을 활용하여 일시적 연결이 끊겨도 데이터 입력이 가능하고, 연결 복구 후 동기화된다.

부록 A. Google Workspace 리소스 구조

이 프로젝트는 별도의 코드 리포지터리 대신 Google Workspace 리소스를 활용한다. 추천 구조는 다음과 같다:

```
Google Drive/
├── EduDocu/
│   ├── templates/          # 서류 템플릿 엑셀 파일 (월간계획서, 출석부 등)
│   ├── generated_docs/     # Apps Script가 생성한 엑셀 파일 저장 위치
│   └── other_assets/       # 지도안·계획서 등 첨부 파일
Google Sheets/
├── edudocu_data.xlsx       # Schools, Classes, Students, Curriculum,
Attendance, Payments, Documents 시트 포함
Apps Script/
└── EduDocuManager.gs       # 공휴일 호출, 날짜 계산, 문서 생성, Gemini API 연동 함수
```

Apps Script 프로젝트는 Google Sheets에 바인딩할 수 있으며, 스크립트 파일(`EduDocuManager.gs`)은 여러 함수와 트리거를 포함한다. 템플릿과 생성된 문서 폴더는 공유 권한을 적절히 설정하여 외부 유출을 방지한다.

본 문서는 에듀도큐 매니저 v1.0 개발을 위한 기준 계획서입니다. 개발 진행에 따라 내용이 업데이트될 수 있습니다.

---

1 When to use AppSheet or Apps Script in Google Workspace | Google Workspace Blog
https://workspace.google.com/blog/developers-practitioners/when-to-use-appsheet-or-apps-script-in-google-workspace

2 Google Sheets as a Database: Pros & Cons
https://stackby.com/blog/google-sheets-database/

3 Comprehensive guide: Export Google Sheets as PDF, Excel, CSV, or other formats using Apps Script
https://spreadsheet.dev/comprehensive-guide-export-google-sheets-to-pdf-excel-csv-apps-script